



پروژه کنترل سطح و دمای آب

تهیه کنندگان:

ارغوان معماری

عرفان فقهی

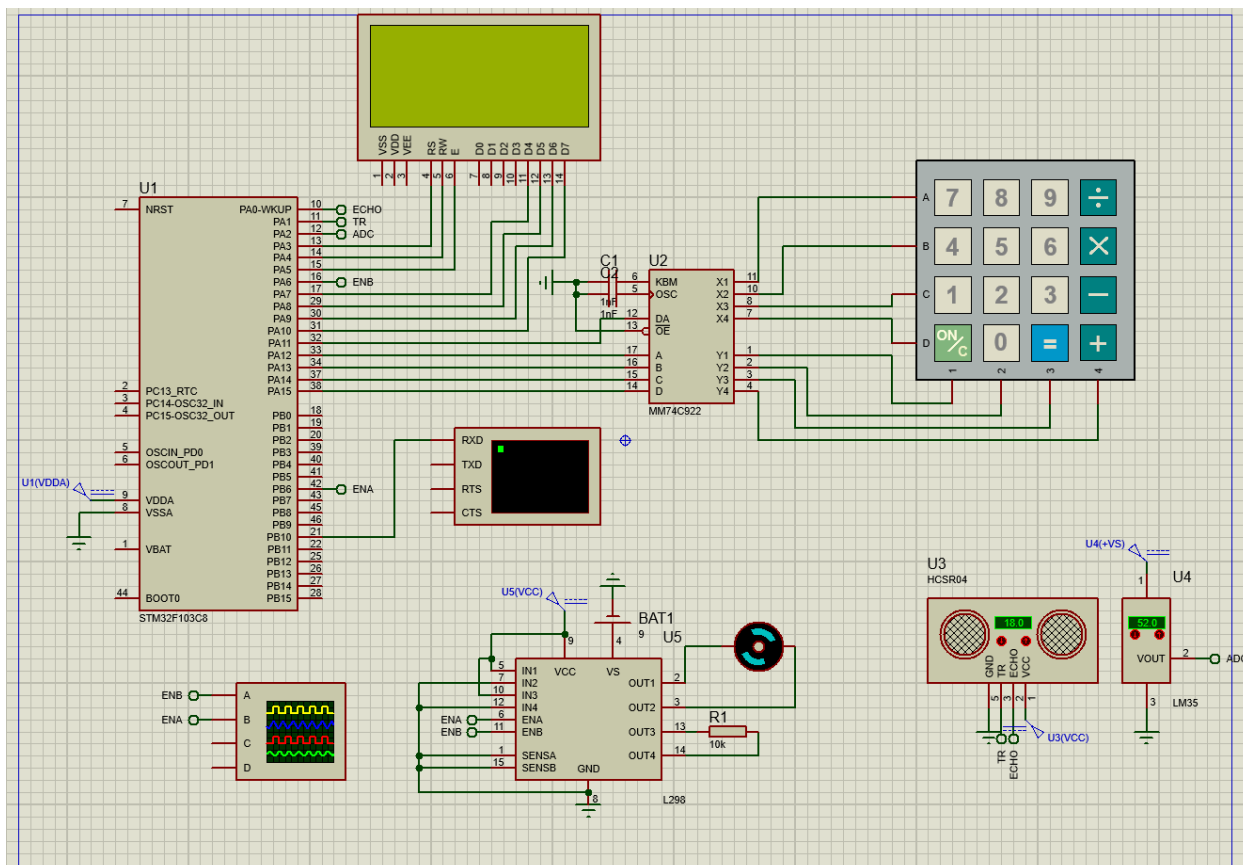
علیرضا منتجب

نیمسال تحصیلی

1404-1405

مقدمه

در این پروژه یک سیستم کنترل سطح مخزن و کنترل دما طراحی شده است. برای اندازه‌گیری سطح از سنسور اولتراسونیک HCSR04 و برای اندازه‌گیری دما از سنسور LM35 استفاده شده است. موتور توسط درایور L298 و سیگنال PWM کنترل می‌شود. همچنین امکان تغییر SetPoint و زمان توسط Keypad فراهم شده است.



معرفی قطعات استفاده شده

توضیحات	قطعه
میکروکنترلر اصلی سیستم که وظیفه پردازش و کنترل را بر عهده دارد.	STM32F103
سنسور اولتراسونیک برای اندازه گیری فاصله سطح آب.	HCSR04
سنسور آنالوگ دما با خروجی 10 mV بر درجه سانتی گراد.	LM35
درایور موتور برای راه اندازی موتور DC با PWM.	L298
نمایش دما، فاصله، ست پوینت و ساعت.	LCD
برای ورود SetPoint و تغییر زمان.	Keypad
هیتر	مقاومت

توضیح عملکرد سیستم

کنترل سطح آب

اندازه‌گیری فاصله سطح آب با استفاده از سنسور HCSR04 و کنترل سرعت موتور پمپ به صورت متناسب با مقدار خطا.

متغیر ها :

IC_Val1 : زمان ثبت لبه بالا رونده سیگنال Echo

IC_Val2 : زمان ثبت لبه پایین رونده سیگنال Echo

Difference : اختلاف زمانی بین دو لبه

Is_First_Captured : مشخص می‌کند اولین لبه ثبت شده یا نه

Distance : فاصله محاسبه شده بر حسب سانتی‌متر

مراحل عملکرد سیستم:

ارسال پالس تحریک به سنسور:

در ابتدا پایه TRIG سنسور HCSR04 برای مدت کوتاهی در وضعیت HIGH قرار می‌گیرد. این پالس باعث ارسال یک موج اولتراسونیک با فرکانس 40 کیلوهرتز به سمت سطح آب می‌شود.

انتشار موج در محیط:

موج اولتراسونیک در هوا حرکت می‌کند تا به سطح آب برخورد کند. پس از برخورد، موج به سمت سنسور بازتاب داده می‌شود.

فعال شدن پایه Echo:

پس از بازگشت موج، پایه Echo سنسور در وضعیت HIGH قرار می‌گیرد. مدت زمانی که این پایه HIGH باقی می‌ماند برابر با زمان رفت و برگشت موج صوتی است.

ثبت زمان لبه بالا رونده:

در اولین تغییر وضعیت Echo از LOW به HIGH، وقفه تایمر فعال شده و مقدار شمارنده تایمر در متغیر IC_Val1 ذخیره می‌شود. این لحظه شروع اندازه‌گیری زمان است.

تغییر قطبیت تایمر:

پس از ثبت لبه بالا رونده، تایمر روی لبه پایین رونده تنظیم می‌شود تا پایان پالس Echo اندازه‌گیری شود.

ثبت زمان لبه پایین رونده:

زمانی که Echo از HIGH به LOW برگردد، مقدار شمارنده تایمر در متغیر IC_Val2 ذخیره می‌شود.

محاسبه اختلاف زمانی:

Difference برابر است با اختلاف IC_Val2 و IC_Val1 این مقدار نشان دهنده مدت زمان رفت و برگشت موج صوتی است.

جبران سرریز تایمر:

اگر شمارنده تایمر سرریز شده باشد، اختلاف زمان با در نظر گرفتن مقدار ماکزیمم تایمر محاسبه می شود.

محاسبه فاصله:

Distance برابر است با Difference تقسیم بر 61.

عدد 61 ضریب تبدیل زمان به سانتی متر است که بر اساس سرعت صوت در هوا و رفت و برگشت موج به دست آمده است.

نمایش فاصله:

مقدار Distance روی LCD نمایش داده می شود تا کاربر بتواند فاصله سطح آب را مشاهده کند.

کنترل موتور:

هدف کنترل:

تنظیم سرعت موتور پمپ متناسب با میزان خطای فاصله.

محدودسازی فاصله:

اگر Distance بیشتر از 50 سانتی متر باشد، مقدار آن برابر 50 در نظر گرفته می شود تا سیستم در محدوده طراحی شده کار کند.

محاسبه خطای فاصله:

خطا برابر است با اختلاف فاصله اندازه گیری شده از مقدار مرجع سیستم.

رابطه خطی تولید: PWM

مقدار diserror با استفاده از یک رابطه خطی بین فاصله اندازه گیری شده (Distance) و مقدار تنظیم شده (SetPoint)

محاسبه می شود. در این روش، هرچه فاصله از مقدار SetPoint کمتر شود، مقدار PWM افزایش می یابد تا سرعت موتور بیشتر شود و سیستم به مقدار مرجع برسد.

```
if(setpoint != 0)
{
    if(Distance > setpoint)
        diserror = 0;
    else if(Distance <= 0)
        diserror = 159;
    else
        diserror = 159 - (Distance * (159 - 0) / setpoint);
}
__HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, diserror);
```

اعمال سیگنال: PWM

با استفاده از دستور

__HAL_TIM_SET_COMPARE

مقدار Duty Cycle تایمر تغییر داده می‌شود و در نتیجه سرعت موتور تنظیم می‌شود.

رفتار سیستم:

Distance کمترین: مقدار خطا بیشینه در نظر گرفته می‌شود و سیگنال PWM در بیشترین مقدار قرار می‌گیرد، بنابراین موتور با حداکثر سرعت کار می‌کند.

Distance برابر با مقدار SetPoint: مقدار خطا صفر در نظر گرفته می‌شود و سیگنال PWM در حداقل مقدار تنظیم می‌شود، بنابراین موتور متوقف شده یا با کمترین سرعت ممکن کار می‌کند.

Distance بین صفر تا: SetPoint

سرعت موتور متناسب با مقدار فاصله به صورت خطی تغییر می‌کند.

نوع کنترل استفاده شده:

نوع کنترل : کنترل تناسبی (Proportional Control)

```
104 //-----HCSR04-----//
105 uint32_t IC_Val1 = 0; //مقدار اول
106 uint32_t IC_Val2 = 0; //مقدار دوم
107 uint32_t Difference = 0; //تفاوت
108 uint8_t Is_First_Captured = 0;
109 int Distance = 0; //فاصله
110
111 void HCSR04_Read(void)
112 {
113     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
114     HAL_Delay(1);
115
116     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET);
117     HAL_Delay(1);
118
119     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
120
121     void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
122     {
123         if (htim->Instance == TIM2 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
124         {
125             if (Is_First_Captured == 0)
126             {
127                 IC_Val1 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
128                 Is_First_Captured = 1;
129             }
130             __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1, TIM_INPUTCHANNELPOLARITY_FALLING);
131         }
132         else
133         {
134             IC_Val2 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
135             if (IC_Val2 > IC_Val1) Difference = IC_Val2 - IC_Val1;
136             else Difference = (0xFFFF - IC_Val1) + IC_Val2;
137
138             Distance = Difference / 61; // 61cm
139
140             Is_First_Captured = 0;
141             __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1, TIM_INPUTCHANNELPOLARITY_RISING);
142         }
143     }
144 }
```

```
//-----Distance-----
HCSR04_Read();
if(Distance>50)
{
    sprintf(distStr, "dist:50cm");
    lcd_gotoxy(0, 0);
    lcd_puts(distStr);
}
else{
    sprintf(distStr, "dist:%dcm", Distance);
    lcd_gotoxy(0, 0);
    lcd_puts(distStr);
}
if(setpoint != 0)
{
    if(Distance > setpoint)
        diserror = 0;
    else if(Distance <= 0)
        diserror = 159;
    else
        diserror = 159 - (Distance * (159 - 0) / setpoint);
}
__HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, diserror);
```

کنترل دما

تنظیم هیتر به گونه‌ای که دمای سیستم به مقدار $SetPoint = 60$ تعیین شده برسد و در آن مقدار پایدار بماند .

مراحل عملکرد سیستم:

خواندن سیگنال آنالوگ سنسور:

دمای محیط توسط سنسور LM35 اندازه‌گیری می‌شود. خروجی این سنسور به ورودی ADC میکروکنترلر متصل است.

تبدیل آنالوگ به دیجیتال:

مقدار ولتاژ خروجی سنسور توسط واحد ADC خوانده می‌شود و به مقدار دیجیتال تبدیل می‌گردد.

محاسبه دمای واقعی:

مقدار خوانده شده در ضرب تبدیل ضرب می‌شود تا دما بر حسب درجه سانتی‌گراد محاسبه شود.

نمایش دما:

دمای اندازه‌گیری شده روی LCD نمایش داده می‌شود تا کاربر مقدار لحظه‌ای را مشاهده کند.

در دمای 40 درجه:

PWM بیشترین مقدار خود را داشته باشد.

در دمای برابر با 60:

PWM به کمترین مقدار خود برسد.

طراحی رابطه کنترلی:

هدف طراحی رابطه:

در این پروژه طراحی به گونه‌ای انجام شده که:

دمای 40 درجه:

temperror بیشترین باشد (حداکثر PWM)

دمای برابر با 60

temperror برابر کمترین باشد (کمترین PWM)

برای رسیدن به این رفتار، از یک رابطه خطی بین دما و مقدار PWM استفاده شده است.

$$\text{temperror} = 7.5 * \text{adc} - 286;$$

رفتار سیستم:

در دمای 40 درجه:

PWM در بیشترین مقدار خود قرار دارد و هیتر با حداکثر توان کار می کند.

در دمای برابر با 60 درجه:

PWM در کمترین مقدار خود قرار می گیرد و هیتر در حداقل توان یا حالت خاموش کار می کند.

اگر دما بین این دو مقدار باشد:

توان هیتر به صورت خطی و متناسب با دما تغییر می کند.

پس از محاسبه `temperror`، مقدار آن به تایمر PWM داده می شود:

`__HAL_TIM_SET_COMPARE`

این دستور Duty Cycle را تغییر می دهد و در نتیجه سرعت موتور تنظیم می شود.

نوع کنترل استفاده شده:

نوع کنترل: کنترل تناسبی (Proportional Control)

```
//-----temp-----  
// گرفتن دما  
HAL_ADC_Start(&hadc1);  
if(HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY)==HAL_OK){  
    adc=HAL_ADC_GetValue(&hadc1);  
    adc=adc*0.0806;  
}  
if(adc > 40 && adc < 60) temperror=0;  
temperror=7.5*adc-286;  
while(adc<40){  
    HAL_ADC_Start(&hadc1);  
    if(HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY)==HAL_OK){  
        adc=HAL_ADC_GetValue(&hadc1);  
        adc=adc*0.0806;  
    }  
    lcd_clear();  
    HAL_Delay(500);  
    lcd_gotoxy(0, 0);  
    lcd_puts("TOO COLD");  
    HAL_Delay(500);  
}  
__HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,159-temperror);  
  
sprintf(adc1,"temp:%2d",adc);  
lcd_gotoxy(0, 2);  
lcd_puts(adc1);
```


طراحی و عملکرد منوی سیستم

هدف طراحی منو:

در این پروژه برای افزایش قابلیت تنظیم سیستم، یک منوی کاربری طراحی شد تا کاربر بتواند بدون تغییر در برنامه، پارامترهای مهم سیستم را تنظیم کند.

این منو از طریق Keypad کنترل می‌شود و اطلاعات روی LCD نمایش داده می‌شود.

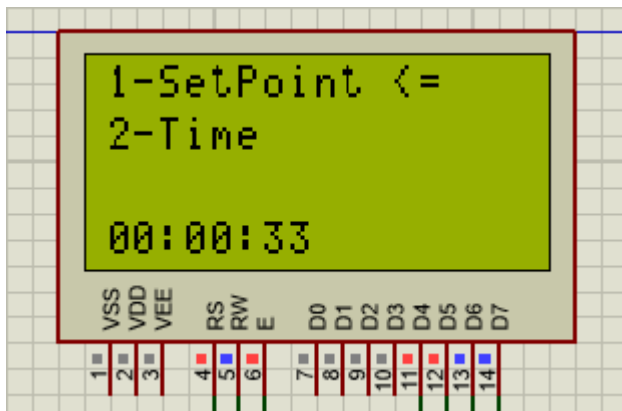
ساختار منو:

منوی اصلی شامل دو گزینه است:

1- SetPoint

2- Time

کاربر با استفاده از کلیدهای جهت‌دار بین این گزینه‌ها جابه‌جا می‌شود و با فشردن کلید "on" گزینه مورد نظر را انتخاب می‌کند.



نحوه ورود به منو:

فشردن کلید: "on"

سیستم از حالت عادی خارج شده و وارد محیط منو می‌شود.

در این حالت صفحه LCD گزینه‌های منو را نمایش می‌دهد.

گزینه اول: تغییر SetPoint

نمایش پیام ورود مقدار:

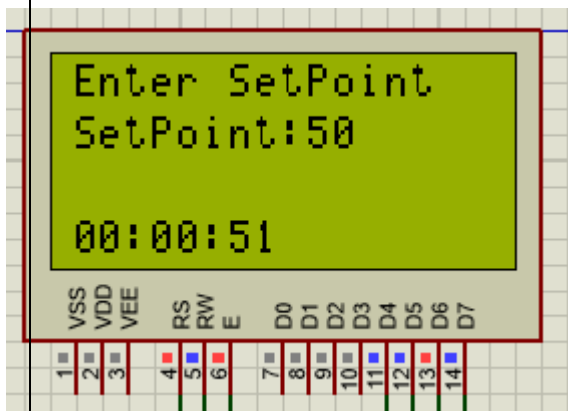
عبارت Enter SetPoint روی LCD نمایش داده می‌شود.

دریافت ورودی عددی:

کاربر با استفاده از Keypad مقدار جدید را وارد می‌کند.

بررسی صحت مقدار:

اگر مقدار وارد شده در بازه مجاز باشد، به عنوان SetPoint جدید ذخیره می‌شود.



ارسال پیام موفقیت:

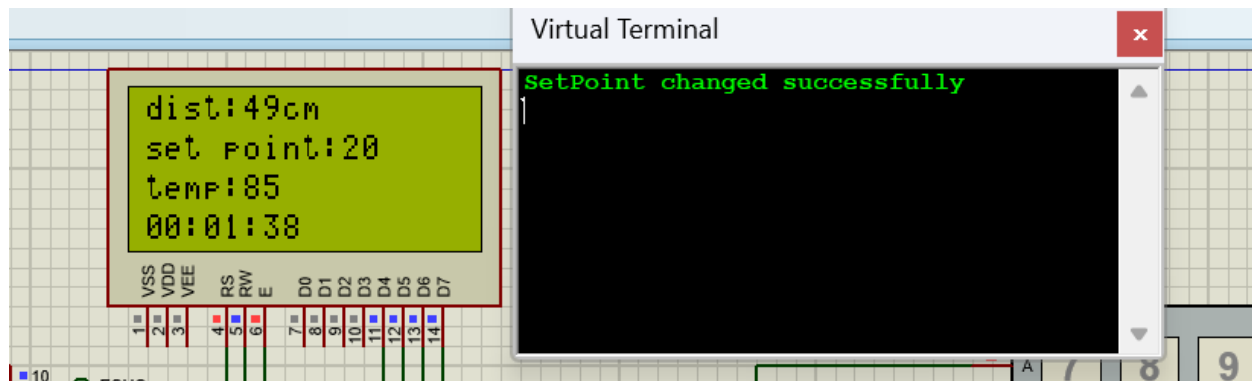
پس از تغییر موفق، پیام

SetPoint changed successfully

از طریق UART ارسال می‌شود.

بازگشت به حالت عادی:

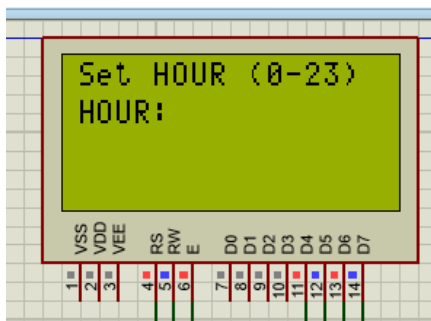
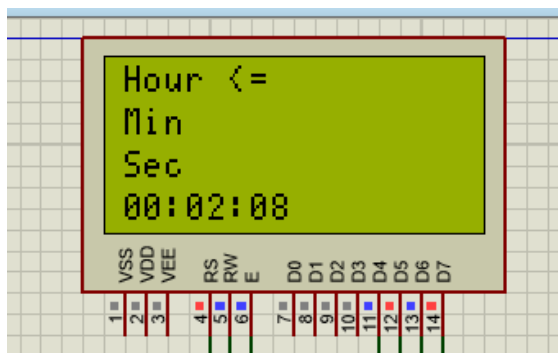
پس از ثبت مقدار، سیستم از منو خارج شده و به حالت اصلی بازمی‌گردد.



گزینه دوم : تغییر زمان RTC

با انتخاب گزینه‌ی **Time** از منوی اصلی، منوی تغییر زمان باز می‌شود. در این منو کاربر می‌تواند به صورت مجزا یکی از سه بخش زمان را تنظیم کند:

- تغییر ساعت (Hour)
- تغییر دقیقه (Minute)
- تغییر ثانیه (Second)

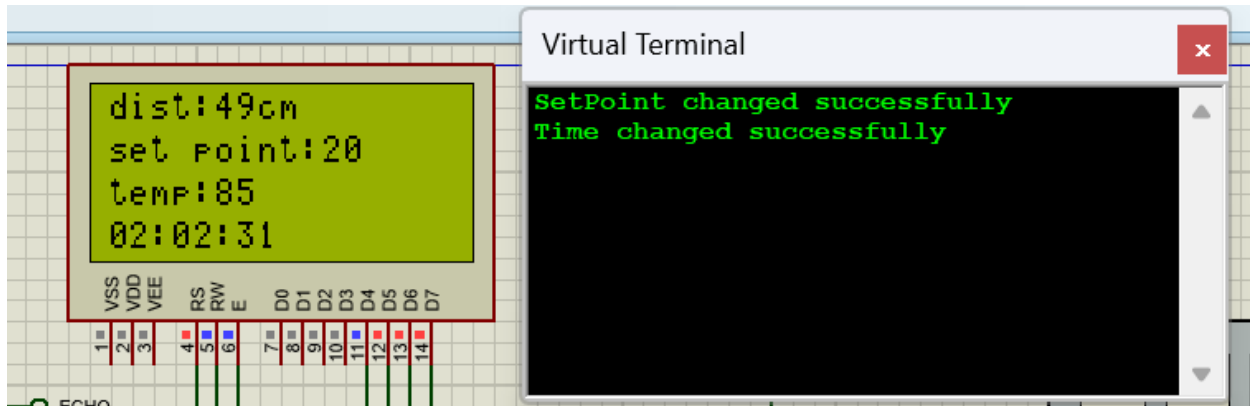


در این طراحی، هر بخش به صورت جداگانه قابل ویرایش است تا کاربر بتواند بدون نیاز به وارد کردن کل زمان، فقط قسمت موردنظر را اصلاح کند. پس از انتخاب هر گزینه، مقدار جدید از طریق کیبورد وارد شده و پس از بررسی صحت مقدار (مثلاً ساعت کمتر از 24 و دقیقه و ثانیه کمتر از 60)، مقدار جدید در ماژول RTC ثبت می‌شود.

ارسال پیام موفقیت:

Time changed successfully

از طریق UART ارسال می‌شود.



رفتار سیستم در منو:

در زمان حضور در منو، ساعت جاری همچنان روی LCD نمایش داده می‌شود.

این کار باعث می‌شود کاربر وضعیت سیستم را همزمان مشاهده کند.

پس از پایان تنظیمات، سیستم به حالت عادی بازمی‌گردد و عملیات کنترل ادامه پیدا می‌کند.

توضیح کد منو

در این پروژه برای تنظیم پارامترهای سیستم، یک منوی ساده مبتنی بر کیپد طراحی شده است. این منو به کاربر اجازه می‌دهد مقادیر مهم سیستم را تغییر دهد.

ساختار منو بر اساس یک ماشین حالت ساده (State Machine) پیاده‌سازی شده است که با دو متغیر اصلی کنترل می‌شود:

- متغیر a برای مشخص کردن مرحله‌ی منو
- متغیر b برای تعیین گزینه‌ی انتخاب شده

ورود به منو

با فشردن کلید "on"، سیستم وارد حالت منو می‌شود. در این حالت:

- صفحه LCD گزینه‌های موجود را نمایش می‌دهد:

1-SetPoint ○

2-Time ○

- مقدار a برابر 1 قرار می‌گیرد، که نشان‌دهنده‌ی مرحله‌ی انتخاب گزینه است.

انتخاب گزینه

در مرحله‌ی انتخاب:

- با فشردن کلید +گزینه‌ی SetPoint انتخاب می‌شود.
- با فشردن کلید -گزینه‌ی Time انتخاب می‌شود.
- با فشردن مجدد کلید "on"، گزینه‌ی انتخاب‌شده تأیید شده و مقدار a برابر 2 می‌شود (ورود به زیرمنو).

زیرمنوی SetPoint

در این بخش:

- کاربر می‌تواند مقدار جدید SetPoint را با استفاده از کیپد وارد کند.
- اعداد واردشده در یک بافر ذخیره می‌شوند.
- پس از فشردن "on" مقدار وارد شده بررسی می‌شود.
- اگر مقدار در محدوده مجاز باشد، SetPoint جدید ثبت می‌شود.
- در صورت موفقیت، پیام تأیید از طریق UART ارسال می‌شود.
- سپس سیستم از منو خارج شده و به حالت عادی باز می‌گردد.

زیرمنوی Time

در این بخش، منوی تغییر زمان شامل سه گزینه‌ی مجزا است:

- تغییر ساعت (Hour)
- تغییر دقیقه (Minute)
- تغییر ثانیه (Second)

پس از ورود به این زیرمنو، کاربر می‌تواند یکی از این سه گزینه را انتخاب کند. سپس مقدار جدید مربوط به همان بخش از طریق کیپد وارد می‌شود.

برنامه مقدار وارد شده را بررسی می‌کند:

- ساعت باید کمتر از 24 باشد.
- دقیقه باید کمتر از 60 باشد.
- ثانیه باید کمتر از 60 باشد.

در صورت معتبر بودن مقدار، فقط همان بخش از زمان در ماژول RTC به‌روزرسانی می‌شود.

در صورت نامعتبر بودن مقدار، پیام خطا روی LCD نمایش داده می‌شود و تغییر اعمال نمی‌گردد.

پس از ثبت موفق یا نمایش خطا، سیستم به منوی قبلی بازمی‌گردد.

خروج از منو

پس از تنظیم هر پارامتر، مقدار a برابر صفر قرار می‌گیرد و برنامه به حلقه اصلی بازمی‌گردد. در این حالت سیستم مجدداً:

- دما
- فاصله
- SetPoint
- زمان

را روی LCD نمایش می‌دهد و کنترل PWM ادامه پیدا می‌کند.

جمع‌بندی

در این پروژه، یک سیستم کنترلی مبتنی بر میکروکنترلر STM32 طراحی و پیاده‌سازی شد که قابلیت اندازه‌گیری دما، فاصله، نمایش زمان و کنترل عملگر از طریق PWM را دارد.

برای افزایش انعطاف‌پذیری سیستم، یک منوی تنظیمات طراحی شد که به کاربر اجازه می‌دهد در زمان اجرا، پارامترهای مهمی مانند **SetPoint** و زمان **RTC** را تغییر دهد. منوی طراحی شده به صورت ساختارمند و مرحله‌ای پیاده‌سازی شده است تا کاربر بتواند به سادگی بین گزینه‌ها جابجا شده و مقدار موردنظر را وارد کند.

در بخش کنترل، با استفاده از روابط خطی مناسب، مقدار خطا (Error) محاسبه شده و بر اساس آن سیگنال PWM تنظیم می‌شود تا عملکرد سیستم متناسب با شرایط تنظیم شده باشد. همچنین برای جلوگیری از ثبت مقادیر نامعتبر، بررسی محدوددهی ورودی‌ها در تمامی بخش‌ها انجام شده است.

در مجموع، این سیستم نمونه‌ای از یک سامانه‌ی کنترلی تعاملی است که هم قابلیت اندازه‌گیری و هم امکان تنظیم پارامترها توسط کاربر را فراهم می‌کند و نشان‌دهنده‌ی تلفیق صحیح مفاهیم ورودی/خروجی، تایمرها، ADC، RTC و UART در یک پروژه‌ی عملی می‌باشد.

